🎯 FocusAnalyser: AI-Based Attention Span & Productivity Monitoring System

> **A real-time, multi-modal computer vision and temporal analytics system that inspects ocular kinematics, gaze orientation, 3D head pose, fatigue indicators, upper-body posture, and environmental objects through webcam video to compute a continuous attention score, measure uninterrupted focus span, diagnose distraction causes, and produce academic-grade productivity reports.**

📑 Table of Contents

1. [Executive Summary & Motivation](#1-executive-summary--motivation)
2. [End-to-End System Architecture](#2-end-to-end-system-architecture)
3. [68-Point Facial Landmark Mapping](#3-68-point-facial-landmark-mapping)
4. [In-Depth Module-by-Module Technical Breakdown](#4-in-depth-module-by-module-technical-breakdown)
    - [4.1 Face & Landmark Localization](#41-face--landmark-localization-analyzersface_detectorpy)
    - [4.2 Eye Analysis & Eye Aspect Ratio (EAR)](#42-eye-analysis--eye-aspect-ratio-ear-analyzerseye_analyzerpy)
    - [4.3 Gaze Direction & Pupil Sclera Analysis](#43-gaze-direction--pupil-sclera-analysis-analyzersgaze_analyzerpy)
    - [4.4 3D Head Pose Estimation via solvePnP](#44-3d-head-pose-estimation-via-solvepnp-analyzershead_pose_analyzerpy)
    - [4.5 Yawn Detection & Mouth Aspect Ratio (MAR)](#45-yawn-detection--mouth-aspect-ratio-mar-analyzersyawn_analyzerpy)
    - [4.6 Drowsiness, Microsleeps & PERCLOS Metric](#46-drowsiness-microsleeps--perclos-metric-analyzersdrowsiness_analyzerpy)
    - [4.7 Upper-Body & Head Posture Analysis](#47-upper-body--head-posture-analysis-analyzersposture_analyzerpy)
    - [4.8 Distraction Object Detection (YOLO)](#48-distraction-object-detection-yolo-analyzersdistraction_detectorpy)
    - [The Mathematical Core: Multi-Modal Attention Scoring](#5-the-mathematical-core-multi-modal-attention-scoring)
    - [Temporal Engine & Continuous Focus Span Tracking](#6-temporal-engine--continuous-focus-span-tracking)

- [Explainable Diagnostics: "Why Did I Lose Focus?"](#7-explainable-diagnostics-why-did-i-lose-focus)
- [Personalized Baseline Calibration Phase](#8-personalized-baseline-calibration-phase)
- [Database Persistence & Long-Term Analytics (SQLite)](#9-database-persistence--long-term-analytics-sqlite)
- [Visual Dashboards: Live HUD & 4-Panel Analytics Report](#10-visual-dashboards-live-hud--4-panel-analytics-report)
- [Academic Division of Work (4-Member Team Allocation)](#11-academic-division-of-work-4-member-team-allocation)
- [Installation, Dependencies & Execution Guide](#12-installation-dependencies--execution-guide)
- [Directory & File Organization](#13-directory--file-organization)

---

1. Executive Summary & Motivation

Traditional software that claims to measure "attention" or "focus" often relies on crude binary assumptions (e.g., *Is a face present in front of the camera?*). Such simplistic implementations suffer from high false-positive rates: a user can stare blankly at a screen while asleep, look down at a smartphone while keeping their face partially in frame, or tilt their head sideways in extreme fatigue without the system noticing.

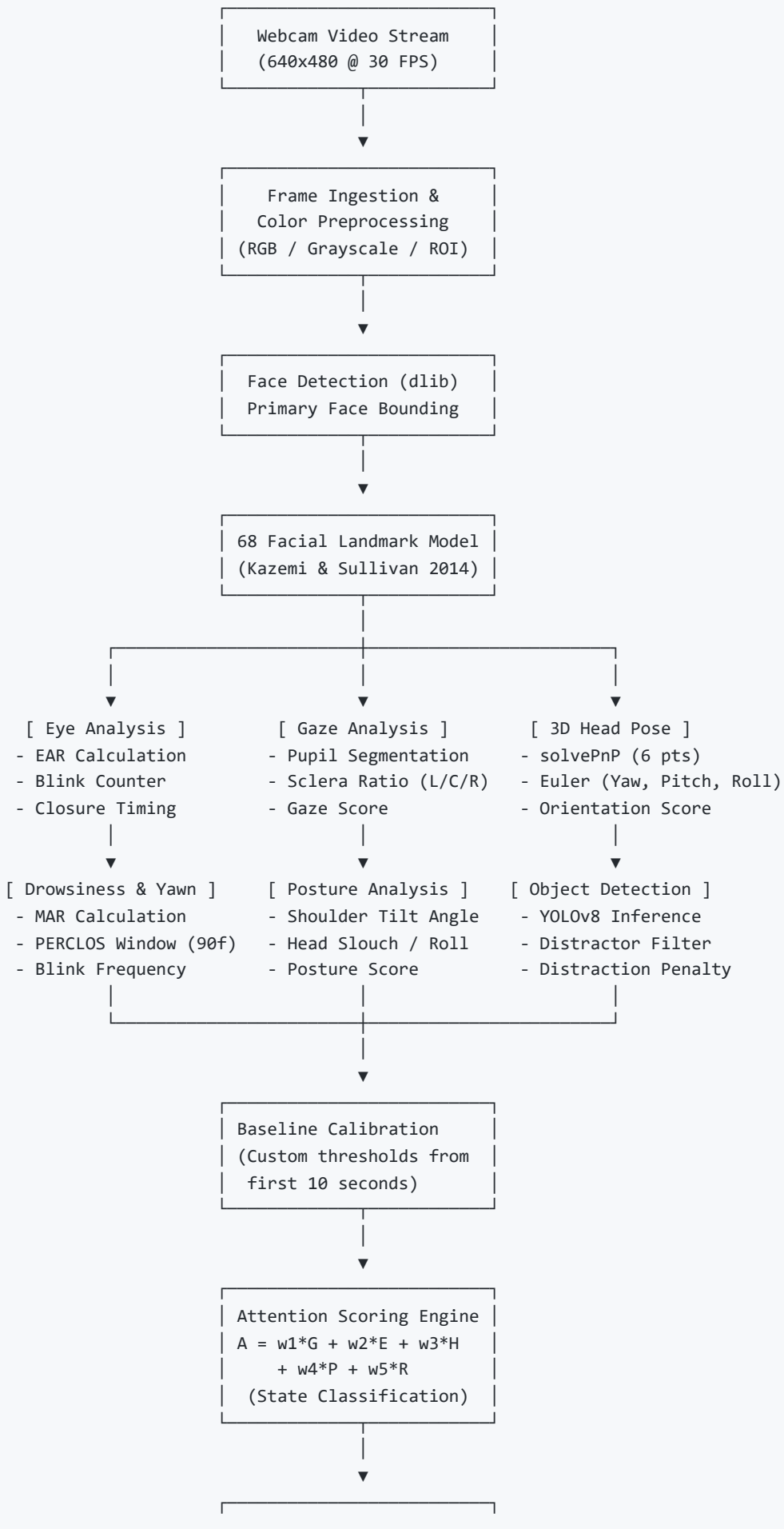
FocusAnalyser treats attention tracking as an **interconnected, multi-modal computer vision and temporal signal-processing challenge**. Instead of treating focus as a static binary state, FocusAnalyser captures video frames at 30 FPS, extracts geometric coordinates of facial features, projects 2D points into 3D metric space to determine spatial head orientation, tracks pupil-to-sclera displacement for gaze classification, applies clinical fatigue metrics (PERCLOS and Mouth Aspect Ratio), and calculates a continuous, weighted **Attention Score ($0\% - 100\%$)**.
Furthermore, attention is mathematically meaningless without a temporal dimension. FocusAnalyser calculates:

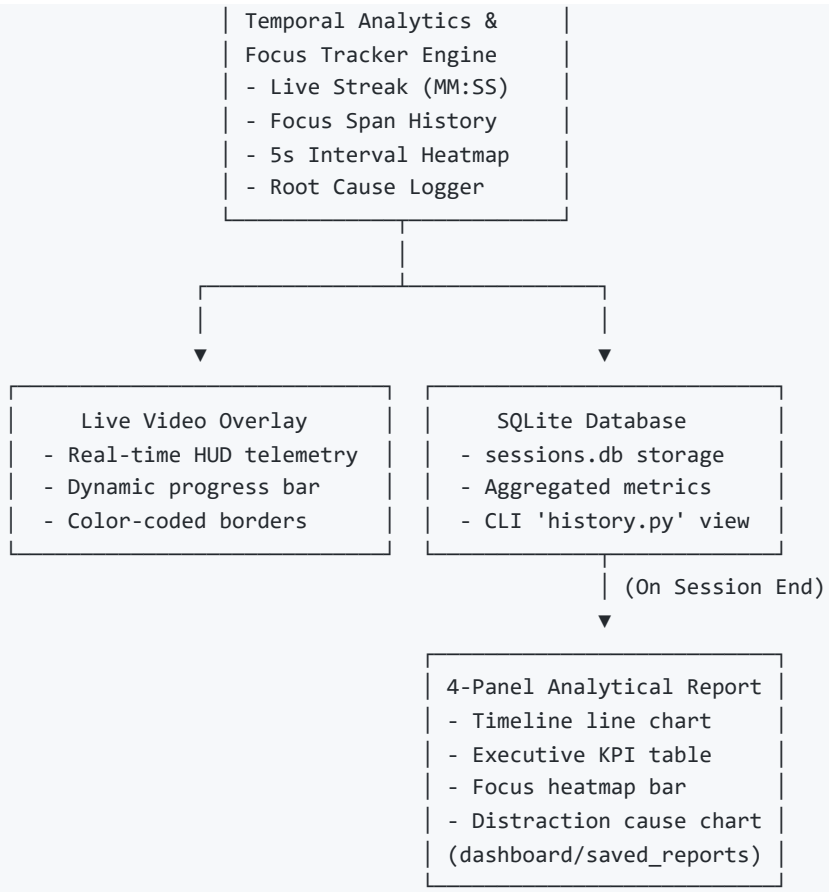
- **Continuous Focus Spans**: Discrete uninterrupted intervals of high concentration.
- **Dynamic Focus Streaks**: Real-time timers that reset with hysteresis when focus lapses.
- **Temporal Heatmaps**: 5-second interval color blocks mapping concentration throughout a study or work session.
- **Root-Cause Attribution**: Explainable logging of *why* concentration lapsed (e.g., phone detected vs. head turned vs. microsleep).

---

2. End-to-End System Architecture

The following diagram illustrates the flow of a single frame through the FocusAnalyser pipeline:

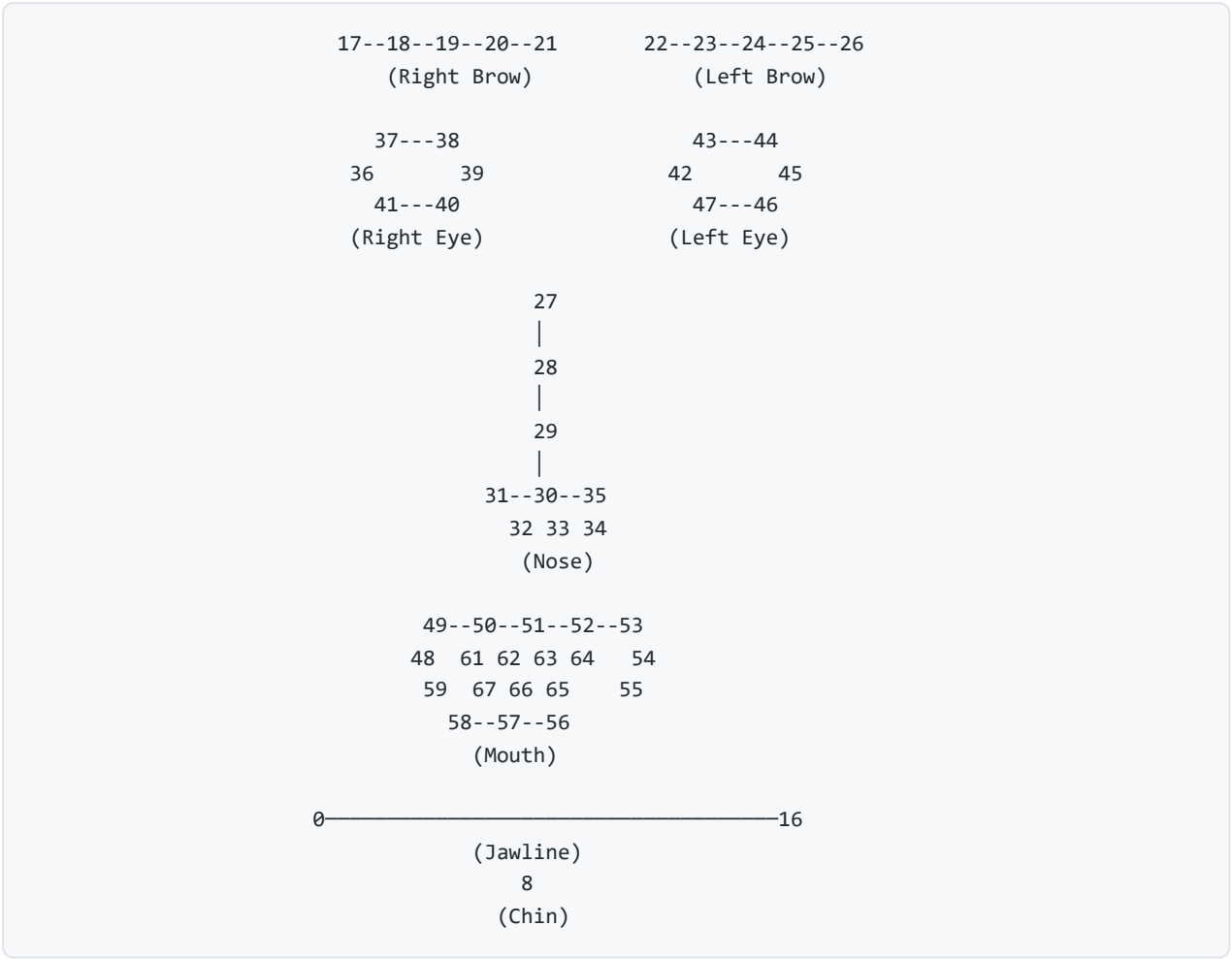

Webcam Video Stream
(640x480 @ 30 FPS)
Frame Ingestion &
Color Preprocessing
(RGB / Grayscale / ROI)
Face Detection (dlib)
Primary Face Bounding
68 Facial Landmark Model
(Kazemi & Sullivan 2014)
[Eye Analysis]
- EAR Calculation
- Blink Counter
- Closure Timing
[Gaze Analysis]
- Pupil Segmentation
- Sclera Ratio (L/C/R)
- Gaze Score
[3D Head Pose]
- solvePnP (6 pts)
- Euler (Yaw, Pitch, Roll)
- Orientation Score
[Drowsiness & Yawn]
- MAR Calculation
- PERCLOS Window (90f)
- Blink Frequency
[Posture Analysis]
- Shoulder Tilt Angle
- Head Slouch / Roll
- Posture Score
[Object Detection]
- YOLOv8 Inference
- Distractor Filter
- Distraction Penalty
Baseline Calibration
(Custom thresholds from
first 10 seconds)
Attention Scoring Engine
A = w1*G + w2*E + w3*H
+ w4*P + w5*R
(State Classification)



---

3. 68-Point Facial Landmark Mapping

FocusAnalyser uses the standardized dlib/iBUG 68-point anthropometric facial landmark indexing scheme:



Landmark Index Groups

- **Jawline**: Points `[0 - 16]`
- **Right Eyebrow**: Points `[17 - 21]`
- **Left Eyebrow**: Points `[22 - 26]`
- **Nose Bridge & Tip**: Points `[27 - 35]` (Nose tip is `30`)
- **Right Eye**: Points `[36 - 41]` (36: Outer corner, 39: Inner corner)
- **Left Eye**: Points `[42 - 47]` (42: Inner corner, 45: Outer corner)
- **Outer Lips**: Points `[48 - 59]` (48: Left corner, 54: Right corner, 51: Top lip, 57: Bottom lip)
- **Inner Lips**: Points `[60 - 67]` (62: Upper inner lip, 66: Lower inner lip)

---

4. In-Depth Module-by-Module Technical Breakdown

4.1 Face & Landmark Localization (`analyzers/face_detector.py`)

- **Underlying Technique**: Histogram of Oriented Gradients (HOG) combined with a linear Support Vector Machine (SVM) for frontal face detection, followed by Kazemi & Sullivan's ensemble of randomized regression trees for sub-millisecond landmark alignment.

- **Execution Flow**:
  1. The frame is downsampled to grayscale.
  2. The detector locates all face rectangles in the frame.
  3. If multiple people appear, the primary user is isolated by selecting the bounding box with the largest spatial area ($\text{Area} = w \times h$), preventing background individuals from hijacking the tracker.
  4. The 68 facial coordinates are extracted and returned as a list of (x, y) integer tuples.

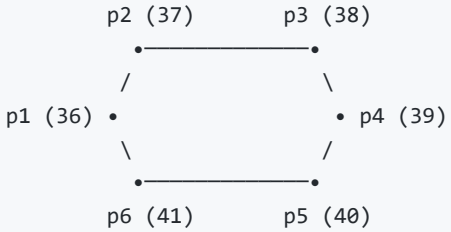
4.2 Eye Analysis & Eye Aspect Ratio (EAR) (`analyzers/eye_analyzer.py`)

Eye openness is quantified using the **Eye Aspect Ratio (EAR)**, introduced by Soukupová and Čech (2016). Rather than running computationally expensive deep neural networks for eye state on every frame, EAR leverages Euclidean distances between the 6 canonical eye landmarks:

$$\text{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \cdot \|p_1 - p_4\|}$$

For the observer's left eye (landmarks `36` through `41`):

- $p_1 = \mathbf{p}_{36}$, $p_4 = \mathbf{p}_{39}$ (Horizontal eye span)
- $p_2 = \mathbf{p}_{37}$, $p_6 = \mathbf{p}_{41}$ (First vertical chord)
- $p_3 = \mathbf{p}_{38}$, $p_5 = \mathbf{p}_{40}$ (Second vertical chord)



Behavior & Derivation:

- When the eye is wide open, the vertical distances are large, yielding $\text{EAR} \approx 0.28 - 0.38$.
- When the eye closes, the numerator drops toward zero while the denominator stays constant, causing EAR to rapidly plummet to ≤ 0.18.
- **Blink Counter State Machine**: When $\text{EAR} < \text{EAR_THRESHOLD}$ for at least $N = 2$ consecutive frames and subsequently recovers above the threshold, a validated blink is recorded. This prevents single-frame noise from triggering false blinks.
- **Continuous Eye Openness Score**:

$$\text{Score}_{\text{eye}} = \begin{cases}

\min\left(1.0, 0.6 + 0.4 \cdot \frac{\text{EAR} - \theta_{\text{EAR}}}{0.15}\right) & \text{if } \text{EAR} \ge \theta_{\text{EAR}} \\

\max\left(0.0, \frac{\text{EAR}}{\theta_{\text{EAR}}} \cdot 0.5\right) & \text{if } \text{EAR} < \theta_{\text{EAR}}

\end{cases}$$

4.3 Gaze Direction & Pupil Sclera Analysis (`analyzers/gaze_analyzer.py`)

Gaze tracking without dedicated infrared hardware relies on **pupil-to-sclera horizontal displacement**:

1. **Region of Interest (ROI) Isolation**: Using landmarks `36-41` and `42-47`, the bounding box surrounding each eye is extracted with a 2-pixel margin.
2. **Noise Reduction**: A 7×7 Gaussian filter is applied to eliminate high-frequency skin texture and digital noise:

$$I_{\text{smooth}}(x, y) = I(x, y) * G_{\sigma}(x, y)$$

1. **Pupil Center Localization**: Because the iris and pupil absorb significantly more light than the surrounding white sclera, the pupil center (x_p, y_p) corresponds to the minimum brightness intensity locus:

$$(x_p, y_p) = \arg\min_{x,y} I_{\text{smooth}}(x, y)$$

1. **Gaze Ratio Computation**:

$$\text{Ratio}_{\text{gaze}} = \frac{x_p - x_{\text{inner}}}{x_{\text{outer}} - x_{\text{inner}}} \in [0.0, 1.0]$$

1. **Classification**:
   - $\text{Ratio} < 0.35 \implies$ **LOOKING LEFT** (Gaze Score = 0.30)
   - $\text{Ratio} > 0.65 \implies$ **LOOKING RIGHT** (Gaze Score = 0.30)
   - $0.35 \leq \text{Ratio} \leq 0.65 \implies$ **CENTER / SCREEN** (Gaze Score = 1.00)

4.4 3D Head Pose Estimation via solvePnP (`analyzers/head_pose_analyzer.py`)

To understand whether the user is facing their monitor or looking away, the system solves the classical **Perspective-n-Point (PnP)** problem. This maps known 3D facial feature coordinates in metric space (mm) to their observed 2D pixel projections on the camera sensor.

1. Anthropometric 3D World Model Points

We define 6 anatomically invariant reference points in metric face space:

- **Nose Tip (`30`)**: $(0.0, 0.0, 0.0)$
- **Chin Tip (`8`)**: $(0.0, -330.0, -65.0)$
- **Left Eye Outer Corner (`45`)**: $(225.0, 170.0, -135.0)$
- **Right Eye Outer Corner (`36`)**: $(-225.0, 170.0, -135.0)$

- **Left Mouth Corner (`54`)**: $(150.0, -150.0, -125.0)$
- **Right Mouth Corner (`48`)**: $(-150.0, -150.0, -125.0)$

2. Camera Intrinsic Calibration Approximation

Using the image dimensions ($W = 640, H = 480$), the pinhole camera matrix $\mathbf{K}$ is modeled assuming an optical center at the frame midpoint:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} W & 0 & W/2 \\ 0 & W & H/2 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{D} = \mathbf{0}_{4\times 1}$$

3. Mathematical Optimization & Euler Angle Extraction

Using Levenberg-Marquardt optimization in `cv2.solvePnP`, the system computes the rotation vector $\mathbf{r}$ and translation vector $\mathbf{t}$ that minimize reprojection error:

$$\arg\min_{\mathbf{R},\mathbf{t}} \sum_{i=1}^{6} \| \mathbf{p}_{2D,i} - \text{project}(\mathbf{K}, \mathbf{R}, \mathbf{t}, \mathbf{P}_{3D,i}) \|^2$$

Using Rodrigues' transformation formula, $\mathbf{r}$ is expanded into a 3×3 orthonormal rotation matrix $\mathbf{R}$:

$$\mathbf{R} = \mathbf{I} + (\sin\theta)\mathbf{K}_\mathbf{r} + (1 - \cos\theta)\mathbf{K}_\mathbf{r}^2, \quad \theta = \|\mathbf{r}\|$$

Finally, `cv2.RQDecomp3x3` decomposes $\mathbf{R}$ into the three intrinsic Euler angles:

- **Yaw (ψ)**: Horizontal rotation left (negative) vs. right (positive). Threshold: $\pm 22^\circ$.
- **Pitch (θ)**: Vertical tilt upward (negative) vs. downward (positive). Threshold: $\pm 18^\circ$.
- **Roll (ϕ)**: Lateral tilt toward left or right shoulder.

4. Head Pose Scoring:

The total angular deviation vector is computed:

$$\Delta_{\text{pose}} = \sqrt{\left(\frac{\psi}{\psi_{\text{thresh}}}\right)^2 + \left(\frac{\theta}{\theta_{\text{thresh}}}\right)^2}$$

$$\text{Score}_{\text{head}} = \begin{cases}

1.0 - 0.30 \cdot \Delta_{\text{pose}} & \text{if } \Delta_{\text{pose}} \le 1.0 \text{ (Facing Screen)} \\

\max(0.20, 0.70 - 0.40 \cdot (\Delta_{\text{pose}} - 1.0)) & \text{if } \Delta_{\text{pose}} > 1.0 \text{ (Turned Away)}

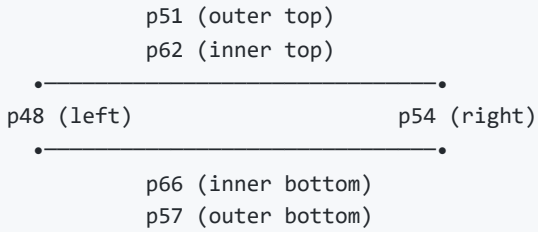
\end{cases}$$

4.5 Yawn Detection & Mouth Aspect Ratio (MAR) (`analyzers/yawn_analyzer.py`)

Drowsiness frequently manifests as involuntary yawning. The **Mouth Aspect Ratio (MAR)** evaluates the ratio of vertical mouth aperture to horizontal mouth span:

$$\text{MAR} = \frac{\|\mathbf{p}_{51} - \mathbf{p}_{57}\| + \|\mathbf{p}_{62} - \mathbf{p}_{66}\|}{2 \cdot \|\mathbf{p}_{48} - \mathbf{p}_{54}\|}$$

- $\mathbf{p}_{51}, \mathbf{p}_{57}$: Outer upper lip center to outer lower lip center.
- $\mathbf{p}_{62}, \mathbf{p}_{66}$: Inner upper lip center to inner lower lip center.
- $\mathbf{p}_{48}, \mathbf{p}_{54}$: Left mouth corner to right mouth corner.



- When talking or resting, $\text{MAR} \approx 0.15 - 0.35$.
- During a genuine yawn, MAR expands to > 0.55.
- **Temporal Filter**: A yawn is only confirmed if $\text{MAR} \geq 0.55$ continuously for ≥ 12 consecutive frames (~0.4s), discarding normal speech patterns or mouth twitches.

4.6 Drowsiness, Microsleeps & PERCLOS Metric (`analyzers/drowsiness_analyzer.py`)

Rather than relying solely on momentary blinks, FocusAnalyser uses **PERCLOS (Percentage of Eye Closure)**—the gold-standard drowsiness metric established by the US Federal Highway Administration (FHWA):

$$\text{PERCLOS} = \frac{1}{W} \sum_{k=t-W+1}^{t} \mathbb{I}(\text{EAR}_k < \theta_{\text{EAR}})$$

- FocusAnalyser computes PERCLOS over a sliding temporal queue of $W = 90$ frames (~3.0 seconds).
- If $\text{PERCLOS} \geq 0.35$ (eyes closed $\geq 35\%$ of the window), an active **Drowsy / Microsleep** alert is triggered.
- **Blink Frequency**: High-frequency shallow blinking ($> 28\ \text{blinks/min}$) indicates ocular fatigue.
- Composite Alertness Score:

$$\text{Score}_{\text{alert}} = \max(0.10, 1.0 - \text{DrowsinessLevel})$$

4.7 Upper-Body & Head Posture Analysis (`analyzers/posture_analyzer.py`)

Ergonomic posture directly correlates with cognitive alertness. FocusAnalyser evaluates posture through a dual approach:

1. **Pose Landmarking (if MediaPipe Pose is present)**:
   - Evaluates left shoulder (`11`) and right shoulder (`12`) coordinates.
   - Calculates shoulder tilt angle $\theta_{\text{shoulder}} = \arctan\left(\frac{|y_2 - y_1|}{|x_2 - x_1|}\right)$.
   - **Facial Geometry Fallback (using 68 dlib landmarks)**:
   - Uses the Roll angle (ϕ) derived from the 3D Head Pose.
   - If $|\phi| > 17^\circ \implies$ **"Head Tilted / Leaning Sideways"** ($\text{Score} = 0.50$).
   - If Pitch $\theta > 18^\circ \implies$ **"Slouching Forward"** ($\text{Score} = 0.45$).
   - Otherwise $\implies$ **"Upright / Good"** ($\text{Score} = 1.00$).

4.8 Distraction Object Detection (YOLO) (`analyzers/distraction_detector.py`)

- Leverages the lightweight **YOLOv8-nano** (`yolov8n.pt`) deep learning model.
- Scans each incoming video frame for common secondary distractor classes:
- `cell phone` , `book` , `bottle` , `cup`
- If a smartphone or distractor is detected with confidence ≥ 0.45:
- A distraction penalty multiplier ($\times 0.30$) is applied directly to the attention score.
- An explainable distraction event (`object_detected`) is logged with a timestamp.

---

5. The Mathematical Core: Multi-Modal Attention Scoring

Instantaneous attention cannot be deduced from a single isolated biometric feature. FocusAnalyser implements a linear multi-attribute utility function:

$$\boxed{A_{\text{raw}} = w_1 \cdot S_{\text{gaze}} + w_2 \cdot S_{\text{eye}} + w_3 \cdot S_{\text{head}} + w_4 \cdot S_{\text{posture}} + w_5 \cdot S_{\text{alert}}}$$

Weight Justification

Symbol	Component	Weight	Justification
S_{gaze}	Gaze Orientation	$w_1 = 0.25$	Primary vector of attention. If the eyes look away from the display, visual attention is broken.
S_{eye}	Eye Openness (EAR)	$w_2 = 0.20$	Confirms sensory input is active (eyes open vs. closed).
S_{head}	3D Head Orientation	$w_3 = 0.25$	Indicates physical orientation toward the workstation.
S_{posture}	Posture Ergonomics	$w_4 = 0.15$	Slouching and heavy tilting correlate with cognitive disengagement.
S_{alert}	Alertness (PERCLOS)	$w_5 = 0.15$	Penalizes fatigue, sluggish blinking, and yawning.
Total		$\sum w_i = 1.00$	Fully normalized $(0.0 \leq A_{\text{raw}} \leq 1.0)$.

Final Score with Distraction Penalty & State Mapping:

$$A = \text{clip}\left(A_{\text{raw}} \cdot P_{\text{distraction}},\ 0.0,\ 1.0\right)$$

Where $P_{\text{distraction}} = 0.30$ if a phone is detected, otherwise 1.0.

$$\text{State}(A) = \begin{cases}

\text{ABSENT} & \text{if no face detected in frame} \\

\text{DROWSY} & \text{if } \text{PERCLOS} \ge 0.35 \text{ or currently yawning} \\

\text{FOCUSED} & \text{if } A \ge 0.60 \text{ } (\ge 60\%) \\

\text{MODERATE} & \text{if } 0.38 \le A < 0.60 \text{ } (38\% - 59\%) \\

\text{DISTRACTED} & \text{if } A < 0.38 \text{ } (< 38\%)

\end{cases}$$

---

6. Temporal Engine & Continuous Focus Span Tracking

Human concentration is a continuous temporal process. An instantaneous drop of 0.2 seconds caused by a sneeze should not count as a lost attention span. FocusAnalyser handles temporal dynamics through `engine/focus_tracker.py` and `engine/temporal_analyzer.py`:

```
Time:     00:00 ──────── 05:00 ──────── 10:00 ──────── 15:00 ──────── 20:00
State:    [ FOCUSED ──────────────] [DIST] [ FOCUSED ──────────────] [DROWSY]
Streak:   0s ──► 300s (Reset!)   0s     0s ──► 300s (Reset!)   0s
Span #1:  |◄── 5.0 mins ──────►|
Span #2:                                 |◄── 5.0 mins ──────►|
```

Key Metrics Tracked:

1. **Live Focus Streak (** 🔥 `MM:SS` **)**:
   - Increments continuously while state is `FOCUSED`.
   - **Hysteresis Reset**: If attention drops into `DISTRACTED` or `DROWSY`, a grace timer begins. The streak only resets if the distraction persists for more than $\Delta t_{\text{reset}} = 2.5$ seconds. This prevents momentary natural blinks from destroying long streaks.
   - **Longest Focus Streak**: The all-time maximum unbroken streak achieved in the session.
   - **Completed Focus Spans**: Every continuous block of concentration > 1.0s is recorded.
   - **Average Focus Span**:

$$\overline{\text{Span}} = \frac{1}{M} \sum_{i=1}^{M} \text{Duration}(\text{Span}_i)$$

1. **Temporal Focus Heatmap**:
   - The session timeline is partitioned into discrete 5.0-second segments.
   - Each segment is classified:
   - 🟢 **HIGH FOCUS** ($\overline{A}_{\text{segment}} \geq 70\%$)
   - 🟡 **MODERATE FOCUS** ($40\% \leq \overline{A}_{\text{segment}} < 70\%$)
   - 🔴 **LOW FOCUS** ($\overline{A}_{\text{segment}} < 40\%$)

---

7. Explainable Diagnostics: "Why Did I Lose Focus?"

Rather than simply telling the user "*You were distracted at 14:32*", the `engine/distraction_logger.py` records the exact combination of underlying physiological and environmental indicators active at the exact moment attention dropped:

```
{
  "timestamp": 872.4,
  "reasons": [
    "head_turned",
    "gaze_away",
    "object_detected (cell phone)"
  ]
}
```

At the end of the session, the system produces a **Contributing Distraction Causes Bar Chart** ranking the frequency of:

- Gaze diverted away
- Head turned away
- Extended eye closure (microsleep)
- Yawning
- Forward slouching / improper posture
- Mobile phone or external object in hand
- User completely absent from camera

---

8. Personalized Baseline Calibration Phase

Every human being has unique resting physiology:

- Some people naturally have smaller or deeper-set eyes (lower baseline EAR).
- Some people sit slightly offset from their webcam.
- Resting blink rates vary from 10 to 25 blinks per minute.

When `main.py` is executed, the first **10 seconds** activate `engine/baseline_calibrator.py`:

1. The user looks comfortably at the center of their screen.
2. The system accumulates sample distributions of resting open-eye EAR, horizontal gaze ratio, and neutral yaw/pitch resting angles.
3. Dynamic personalized thresholds are computed:

$$\theta_{\text{EAR, custom}} = \text{clip}\left(\mu_{\text{EAR}} \cdot 0.72,\ 0.18,\ 0.30\right)$$

$$\theta_{\text{Gaze, min}} = \mu_{\text{gaze}} - 0.18, \quad \theta_{\text{Gaze, max}} = \mu_{\text{gaze}} + 0.18$$

1. This eliminates false alarms caused by anatomical differences.

---

9. Database Persistence & Long-Term Analytics (SQLite)

Every session is permanently stored in `data/sessions.db` using SQLite (`database/session_db.py`).

Relational Schema (`sessions` Table)

```
CREATE TABLE IF NOT EXISTS sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT,                      -- e.g. "2026-10-20"
    start_time TEXT,                -- e.g. "17:15:02"
    duration REAL,                  -- Total session seconds
    attention_score REAL,           -- Overall session score (0 - 100%)
    focused_time REAL,              -- Seconds in focused state
    distracted_time REAL,           -- Seconds in distracted state
    drowsy_time REAL,               -- Seconds in drowsy state
    avg_focus_span REAL,            -- Average focus span in seconds
    longest_focus_span REAL,        -- Record unbroken streak in seconds
    distraction_count INTEGER,      -- Number of distraction events
    blink_count INTEGER,            -- Total blinks observed
    yawn_count INTEGER,             -- Total yawns observed
    primary_distractor TEXT,        -- Leading cause of distraction
    experiment_label TEXT           -- e.g. "Standard", "Phone-Nearby"
);
```

Inspecting History

Users can inspect their historical performance and weekly trends anytime via `history.py`:

```
python history.py
```

Output:

```
================================================================================
  FocusAnalyser — Historical Analytics & Session Log
================================================================================
  Total Sessions: 5  |  Avg Attention: 78.4%  |  Best Session: 88.0%  |  Total Focus: 1.42 hrs
--------------------------------------------------------------------------------
ID   Date        Start     Duration  Score   Focus(s)  Dist(s)  Streak(s)  Label
--------------------------------------------------------------------------------
#5   2026-10-20  17:15:02  342.1     82%     280.4     52.1     184.2      Standard
#4   2026-10-20  16:40:11  612.0     74%     450.2     120.5    140.0      Phone-Nearby
================================================================================
```

---

10. Visual Dashboards: Live HUD & 4-Panel Analytics Report

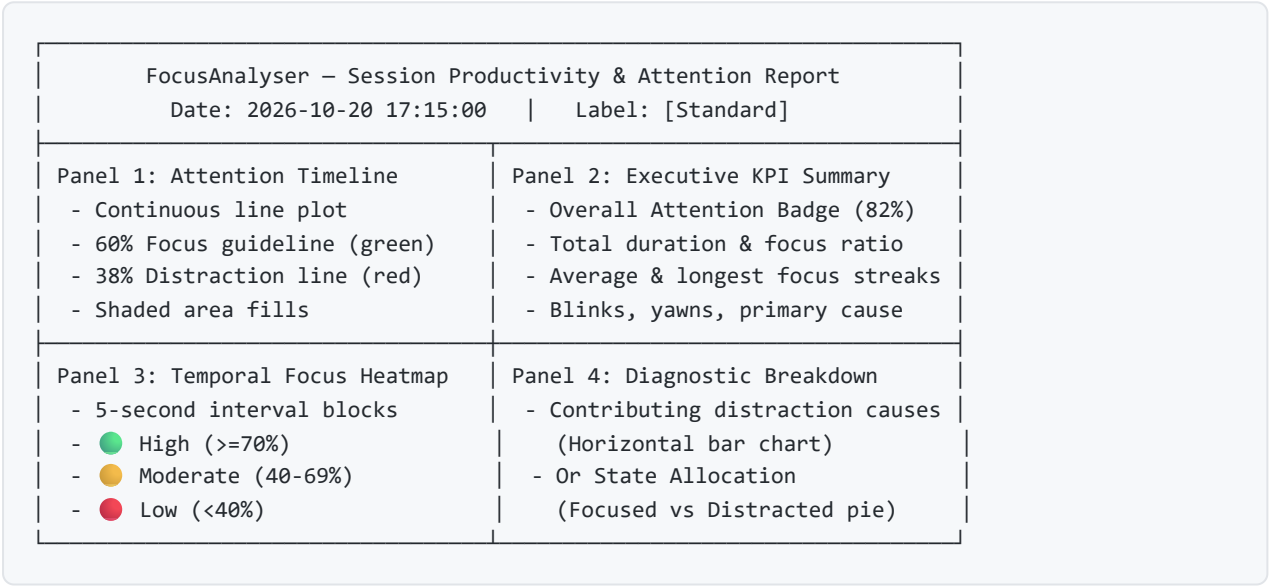
1. Real-Time OpenCV Video HUD (`dashboard/live_overlay.py`)

- **Perimeter Glow**: The entire border of the webcam feed dynamically glows 🟢 Green (Focused), 🟡 Amber (Moderate), 🔴 Red (Distracted), or 🟣 Purple (Drowsy).
- **Top-Left**: Large real-time **Attention Score (%)** readout with a continuous progress bar.
- **Top-Right**: Live **Focus Streak (** 🔥 `MM:SS` **)** counter with your personal record streak.

- **Bottom Telemetry Glass Panel**: Displays exact numerical values for:
- Focused / Distracted / Drowsy elapsed timers.
- Gaze vector (`CENTER` , `LOOKING_LEFT` , `LOOKING_RIGHT`).
- Eye openness state and numeric EAR (`OPEN (0.29)`).
- 3D Head orientation and yaw angle (`FORWARD (Yaw: 3.2°)`).
- Posture status (`Good` , `Head Tilted` , `Slouching Forward`).

2. Publication-Quality 4-Panel Analytical Report (`dashboard/report_generator.py`)

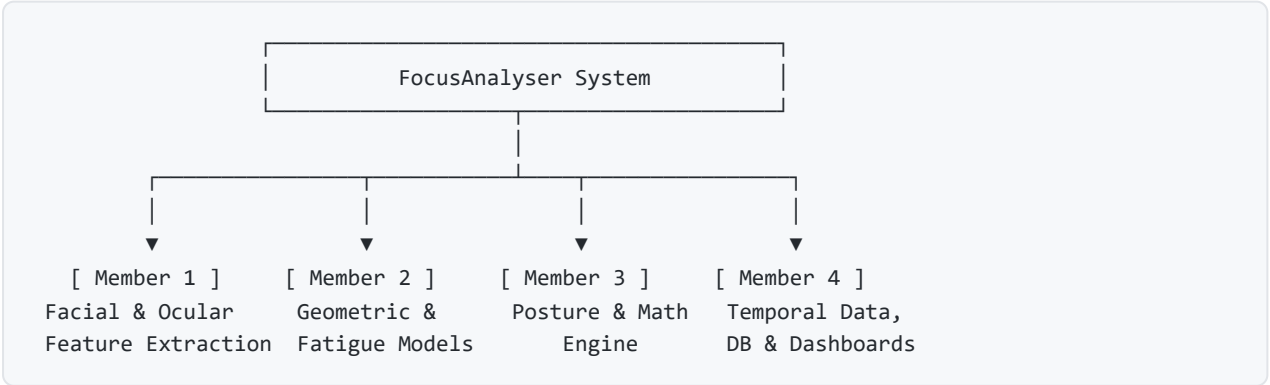
When the user presses `q` , the system computes full session aggregations and generates a high-resolution dark-themed analytical graphic saved automatically to `dashboard/saved_reports/focus_report_YYYY-MM-DD_HH-MM-SS.png` :



---

11. Academic Division of Work (4-Member Team Allocation)

For university evaluations, viva examinations, and project reports, the modules map to 4 distinct engineering specializations:



Member 1: Facial & Ocular Feature Extraction

- **Modules**: [`face_detector.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/face_detector.py), [`eye_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/eye_analyzer.py), [`gaze_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/gaze_analyzer.py)
- **Technical Responsibilities**:
- Image preprocessing, grayscale conversion, and primary face ROI extraction.
- Implementation of the 68-landmark Kazemi-Sullivan regression tree pipeline.
- Mathematical formulation and implementation of Eye Aspect Ratio (EAR).
- Sclera-pupil Gaussian blurring, minimum intensity extraction, and horizontal gaze ratio classification.

Member 2: Geometric Modeling & Fatigue Analysis

- **Modules**: [`head_pose_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/head_pose_analyzer.py), [`yawn_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/yawn_analyzer.py), [`drowsiness_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/drowsiness_analyzer.py)
- **Technical Responsibilities**:
- 3D anthropometric face modeling and camera intrinsic matrix formulation.
- Implementation of `cv2.solvePnP` optimization and Rodrigues rotation transformation.
- RQ decomposition into Euler angles (Yaw, Pitch, Roll) and spatial orientation scoring.
- Mouth Aspect Ratio (MAR) formulation and sliding-window PERCLOS fatigue computation.

Member 3: Posture, Distractions & Mathematical Engine

- **Modules**: [`posture_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/posture_analyzer.py), [`distraction_detector.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/analyzers/distraction_detector.py), [`attention_scorer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/engine/attention_scorer.py), [`focus_tracker.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/engine/focus_tracker.py)
- **Technical Responsibilities**:
- Upper-body tilt and slouching detection using MediaPipe Pose and roll/pitch kinematics.
- Integration of YOLOv8-nano inference for smartphone and distractor detection.
- Multi-modal attention linear utility function: $A = \sum w_i S_i$.
- Continuous focus span tracking, streak timing, and reset hysteresis.

Member 4: Temporal Analytics, Database & Visual Dashboards

- **Modules**: [`temporal_analyzer.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/engine/temporal_analyzer.py), [`baseline_calibrator.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/engine/baseline_calibrator.py), [`session_db.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/database/session_db.py), [`live_overlay.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/dashboard/live_overlay.py), [`report_generator.py`](file:///c:/Users/saura/OneDrive/Desktop/ipcv_project/Focus-Analyser-Attention-Span-Calculator-/dashboard/report_generator.py)
- **Technical Responsibilities**:
- 5-second interval temporal heatmap discretization and moving-average smoothing.
- Statistical personal baseline calibration algorithm.
- SQLite database architecture, schema migration, and CLI historical viewer.
- Real-time OpenCV HUD rendering and Matplotlib 4-panel analytical report generator.

---

12. Installation, Dependencies & Execution Guide

1. Prerequisites

- Python 3.9, 3.10, or 3.11 (64-bit).
- Built-in or external USB Webcam.

2. Install Required Python Packages

```
pip install opencv-python dlib-bin matplotlib scipy numpy
```

(Optional: If you wish to enable smartphone detection, install YOLO: `pip install ultralytics` *)*

3. Verify Model File

Ensure the 68-face landmarks model is present:

- File: `shape_predictor_68_face_landmarks.dat` (~95 MB)
- If missing, download it from dlib's repository:

```
python -c "import urllib.request; urllib.request.urlretrieve('https://github.com/italojs/facial-
```

---

4. Running the Application

Standard Execution:

```
python main.py
```

Running Experiments for Academic Comparison:

Run sessions under different experimental conditions to generate comparative data for your thesis or viva presentation:

```
# Experiment 1: Baseline focused study
python main.py --label "Normal-Study"

# Experiment 2: Smartphone placed on desk
python main.py --label "Phone-Nearby"

# Experiment 3: Background audio / conversation active
python main.py --label "Auditory-Distraction"
```

Optional Flags:

- `--camera 0` : Specify webcam device index.
- `--no-calibrate` : Skip initial 10s calibration phase.
- `--no-gui-chart` : Save report image silently without opening the matplotlib popup window.

---

5. Keyboard Shortcuts During Live Video Feed

- `q` : End the monitoring session, commit record to SQLite, and display the 4-panel analytical report.
- `c` : Force recalibrate personal baseline thresholds.

---

6. Inspecting Past Session Logs & Statistics

```
python history.py
```

---

13. Directory & File Organization

```
Focus-Analyser-Attention-Span-Calculator-/
│
├── main.py                          # Unified entry point & real-time loop orchestrator
├── config.py                        # Centralized thresholds, weights, and color constants
├── history.py                       # CLI utility for historical session analytics
├── requirements.txt                 # Project dependencies list
├── README.md                        # Complete technical documentation
│
├── shape_predictor_68_face_landmarks.dat  # Pretrained 95MB dlib landmark model
│
├── analyzers/                       # Computer Vision Analyzers Layer
│   ├── __init__.py
│   ├── face_detector.py             # Dlib HOG face detector + landmark extractor
│   ├── eye_analyzer.py              # Eye Aspect Ratio (EAR) & blink state machine
│   ├── gaze_analyzer.py             # Pupil-to-sclera horizontal gaze direction
│   ├── head_pose_analyzer.py        # solvePnP 3D pose, Rodrigues, Yaw/Pitch/Roll
│   ├── yawn_analyzer.py             # Mouth Aspect Ratio (MAR) fatigue detection
│   ├── drowsiness_analyzer.py       # Sliding-window PERCLOS & microsleep detection
│   ├── posture_analyzer.py          # Upper body posture, head tilt, forward slouching
│   └── distraction_detector.py      # YOLOv8 deep learning distractor detection
│
├── engine/                          # Intelligence & Temporal Engine Layer
│   ├── __init__.py
│   ├── attention_scorer.py          # Multi-modal attention formula & state mapping
│   ├── focus_tracker.py             # Continuous focus spans, live streak timer, resets
│   ├── temporal_analyzer.py         # 5s interval heatmap segments & curve smoothing
│   ├── baseline_calibrator.py       # 10s personalized anatomical calibration
│   └── distraction_logger.py        # Explainable root-cause diagnostic logger
│
├── database/                        # Persistence Layer
│   ├── __init__.py
│   └── session_db.py                # SQLite session manager (CRUD operations)
│
├── dashboard/                       # Visualization & Reporting Layer
│   ├── __init__.py
│   ├── live_overlay.py              # OpenCV real-time HUD telemetry & gauges
│   ├── report_generator.py          # Publication-quality 4-panel analytical chart
│   └── saved_reports/               # Auto-saved timestamped PNG session reports
│
└── data/                            # Persistent Data Storage
    └── sessions.db                  # SQLite database file
```

📜 Academic References & Citations

1. **Kazemi, V., & Sullivan, J. (2014)**. *One millisecond face alignment with an ensemble of regression trees*. In Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp. 1867-1874.

2. **Soukupová, T., & Čech, J. (2016)**. *Real-Time Eye Blink Detection using Facial Landmarks*. In 21st Computer Vision Winter Workshop (CVWW).
3. **Wierwille, W. W., et al. (1994)**. *Research on Vehicle-Based Driver Status/Performance Monitoring; Development, Validation, and Refinement of Algorithms*. National Highway Traffic Safety Administration (NHTSA) Final Report.
4. **Lepetit, V., Moreno-Noguer, F., & Fua, P. (2009)**. *EPnP: An accurate O(n) solution to the PnP problem*. International Journal of Computer Vision (IJCV), 81(2), 155-166.